The Role of Artificial Intelligence (AI) in Software testing – A Systematic Literature Review

Rajat Sharma[1] and Shahid Ali[2]

[1]Department of Computing, AGI Education, Auckland, New Zealand

2508-043@studentaka.ac.nz

[2]Department of Computing, AGI Education, Auckland, New Zealand

Shahid.ali@aka.ac.nz

Abstract

In today's fast-changing software landscape, the need for effective software testing has grown increasingly vital for guaranteeing quality, reliability, and security in the software-driven world. With the increasing capabilities of Artificial Intelligence (AI), it has the promise of overcoming the known shortcomings of traditional testing methods that are still largely manual, rule-based, and reactive in their approach to quality assurance. The scope of research conducted on AI tools in testing has covered diverse areas such as machine learning, deep learning, natural language processing, and generative AI, showing the potential of these tools in different testing tasks, and identifying some ongoing challenges. In this paper, we'll discuss how traditional and AI-powered tools are used in four critical areas test management, test case management, defect management, and version management and our study results prove that AI-powered testing is better than traditional testing in each of these areas.

Keywords

Artificial Intelligence (AI), Software testing. Test management, Test cases management, Defects management, Version management.

1. Introduction

The application of software systems in various industries has revolutionized the way technology-based applications are developed, delivered and maintained. With the complexity of software has come the need for effective testing to ensure reliability, functionality and security. The traditional testing methods, which rely on a high degree of manual work and a few rules to automate testing, are no longer enough in today's complex and agile software development environment [1]. To counter this, Artificial Intelligence (AI) technologies have come up as a gamechanger in software testing, enhancing the ability to test faster, more extensively and with greater precision than was possible with traditional software testing tools. To counter this, Artificial Intelligence (AI) technologies are emerging as a gamechanger in software testing, offering the promise to enhance speed, scope and precision over traditional software testing tools.

This research is significant because of the urgent need to create and evaluate the current body of knowledge on the use of AI in software testing. Businesses today are facing the challenge of delivering software solutions faster and faster, along with maintaining high quality standards, especially in the modern software development organizations across the globe [3]. AI methods like Machine Learning (ML), Natural Language Processing (NLP), and deep learning have demonstrated tremendous potential for automating complex test activities like test case generation, defect prediction and regression test prioritisation. If there is no complete knowledge of the effectiveness of these techniques in the testing scenario, practitioners may be tempted to invest in tools that are not suitable for their testing situation, leading to poor identification of quality opportunities and poor resource allocation. This study thus

addresses a significant gap in the existing literature by comprehensively summarizing the current state of research into the application of AI in software testing, expanding the scope of both the academic and the industry communities.

To achieve the research objectives a systematic literature review (SLR) approach is used in accordance with the well-established work [4]. The SLR is a systematic, rigorous and auditable process that allows identification, appraisal and synthesis of the available evidence on a given research question. SLRs use fixed inclusion and exclusion criteria, fixed search terms, and structured quality appraisal protocols, and unlike narrative reviews, are not vulnerable to selection bias [4]. This research is aimed to explore the answers to the following questions:

- What are the current uses of AI in software testing and which testing activities are most focused on?
- What are the claimed benefits and drawbacks of AI test tools and techniques versus traditional testing?
- What are the differences between AI-powered software testing and traditional manual and scripted automation-based testing in terms of the tools they use to identify defects, test coverage, and test execution efficiency?
- What are the future directions and trends in the use of AI in software testing?

This research paper is structured as follows. Section 2 presents a literature review on the role of AI in software testing. Section 3 examines the application of AI across various software testing domains. Section 4 discusses the advantages and challenges associated with AI-driven testing. Section 5 compares AI-based approaches with conventional software testing methods. Section 6 outlines emerging trends and future research directions. Section 7 provides the concluding remarks.

2. Literature Review

Several prior studies have examined the role of artificial intelligence in software testing, and the key contributions from this body of work are outlined below.

A study was conducted regarding the importance of continuous integration delivery and deployment to identify the tools, challenges and identifying the gaps in this regard [3]. In this research the thematic analysis method was applied for reviewing various research papers to analyse aspects such as reducing build and test time in CI, increasing awareness on build and test results in CI and raising concerns about the security and scalability in deployment pipeline. Further the study highlighted how the software system needs to be architecture to support the continuous integration and deployment.

The current survey and state of the art of automated test case generation were studied [1]. The research found that manual effort would be reduced and software coverage and testing productivity would be improved through automation. Most importantly, the study found that the traditional methods of automation are based on rules and are expert dependent, which makes it hard to meet changing software needs something AI can easily do.

Another study was done that concentrates on how to utilize machine learning for automation of different activities in software testing [2]. The research emphasizes that the most common application of machine learning algorithm is test case generation, refinement, and evaluation. The researchers found a need for more empirical research to assess the effectiveness of machine learning algorithms to automate testing tasks.

A study was carried out and the review of the software fault prediction using ML was conducted [5]. The study demonstrated that various classifiers like decision tree, neural network, Bayesian classifiers and support vector machine (SVM) can be used to classify fault-prone software components. The study concluded that, in addition to the availability of historical project data, its quality is highly influential for predictive accuracy, an issue that is still present in AI adoption in organisations where the defect repository is not well developed.

A study was conducted to explore regression test prioritisation in an industrial context that uses ML [6]. The study demonstrated that data driven prioritisation could decrease the effort required for regression testing without compromising the effectiveness in identifying faults. Having reliable historical test data was seen as a crucial aspect of successful implementation of AI-based prioritisation.

The more software test coverage (STC) will be enhanced, more opportunities will be given to identify defects in software deployment [7]. In this context, a Many-Objective Optimisation (MO) technique was proposed that improves the coverage of the branches in Software Testing. The experimental results indicated that the proposed approach was capable of covering more percentage of branches and could discover more complex execution path in software system in an efficient way. The study showed the effectiveness of the AI-based optimization techniques to enhance the effectiveness of testing, thus contributing to software quality.

A fully automated testing framework, Sapienz approach, that uses AI to test Android applications was introduced [8]. The framework employed to automatically generate, run and validate software test cases used search-based optimisation techniques. The results demonstrated the ability of AI to automate complex testing tasks and improve the effectiveness of fault detection.

A study was carried out on Continuous Integration, Continuous Delivery and Continuous Deployment practices [3] in order to systematically review these practices. The researcher deployed the maticanalysis method for data analysing that resulted in identifying the approaches and associated tools which facilitate in deploying continuous practices. It was found that the organisations are adopting regular testing and ongoing quality assurance as part of their quick testing and frequent releases. Meanwhile, test automation was seen as a vital component to ensure CI/CD success.

Additionally, a study was carried out to gain more insights regarding the development of AI technologies role in software testing and its advantages [9]. The study covered some of the new applications of AI and opportunities for smart testing approaches that can make testing more efficient, reduce tedious manual work and allow for constant software delivery.

A systematic review on the application of Large Language Models (LLMs) in software engineering and software testing was conducted [10], where they found that Large Language Models (LLMs) can help to substantially reduce manual effort, increase testing productivity and even automatically generate testing artefacts. AI-generated content challenges were also identified in the study, such as issues with reliability, hallucination, transparency, and validation.

A research was conducted to gain an understanding of generative AI technologies and the part they can play in software testing [11]. This study focused on the potential of generative AI models in test case generation, test data generation, defect identification and providing test documentation. The researchers concluded that generative AI shows great promise in software testing but still requires more research to create robust testing validation processes and governance structures.

As aforementioned the potential of AI in the Software Testing process is immense, from test automation to test generation, test prioritisation to intelligent automation and quality assurance etc. They also discuss the advantages of AI for enhancing the efficiency of testing, software quality, and decision-making. But there are still a number of questions that have not been answered in previous studies. *Firstly*, it is not known how much attention is currently being paid to which parts of the software testing life cycle, and what areas are most focused on. *Secondly*, even though several studies have pointed to the advantages of the AI-based testing methods, the practical aspects and associated challenges and limitations of application in industrial settings have not received much attention. *Thirdly*, there is limited evidence for defect identification, test coverage and test execution efficiency, comparing the AI-based enhanced test and the traditional manual and scripted-based test approaches. *Finally*, though, while there have been a lot of breakthroughs in Generative AI and Large Language Models in recent times, the future of generative AI and the impact it can have on software testing remains unclear. The lack of answers to these questions highlights the need for a systematic review to

identify the existing applications, benefits and limitations, effectiveness, and future direction of the use of AI in software testing.

3. AI Applications in Software Testing

As organisations seek to maximize software quality, minimize testing effort, and expedite delivery, the use of AI has become more widespread in software testing. Current approaches based on manual work and predetermined automation scripts do not allow for capturing the complexity of today's software systems. As Agile, DevOps and CI/CD practices take hold, the need for quicker and smarter testing has grown. Testing and quick feedback loops are needed to enable frequent releases in today's software development environment [3]. To address these needs, various AI tools such as ML, Deep Learning (DL), NLP, and Generative AI are being adopted in software testing environments.

AI can help testing systems to interpret patterns in historical data, predict future results and make decisions automatically, without human involvement [2]. AI is transforming SQA by enhancing the way software is tested, making it more intelligent and data driven [9]. The current use cases of AI in software testing can be categorized into four main areas: Test Management, Test Case Management, Defect Management, and Version Management. Each of these subsections discusses the use cases associated with these areas.

Test management includes planning, coordinating, monitoring, execution, and reporting test activities during the Software Development Life Cycle (SDLC). The traditional way to allocate resources, schedule them, and assess risk is for test managers to have experience, manual analysis, and expert judgment. As software becomes more complex, these methods have been found to be increasingly ineffective.

The evolution of AI has mostly been manifested in predictive analytics and intelligent decision support for test management. The AI-powered testing systems can handle massive testing data and supply valuable insights for managerial decisions, revealing trends in defect logs, test results, and software metrics which might be undetectable for manual review [9].

One of the major use cases of AI in test management is effort estimation. By analysing project size, complexity, defect density, and historical data, it was shown that ML models can provide more accurate testing effort estimates than traditional approaches [12]. It is imperative that the estimation is done accurately, or the defects will get "away" from the test; and over-test will use up project resources.

AI is also a key contributor to risk-based test management. Recently, ML models were utilized to analyse software defects histories, software metrics, and change logs, to identify fault-prone modules [2]. It has also been shown that the data-driven prioritisation can help high-risk tests to be identified, which would help the team to focus resources on the most impactful areas [6]. This can help to lower testing workload without compromising software quality.

Another AI contribution are predictive quality analytics. The ML algorithms can use information found in source code repositories, issue trackers, automated testing systems and CI/CD pipelines to forecast software problems before they occur [13]. This proactive approach is especially beneficial in Agile and DevOps environments where quick decisions are needed.

AI tools have also helped with monitoring and reporting. With the help of the AI-powered reporting systems, testing data can be automatically compiled, and real-time dashboards can be created, easing the burden on administration and allowing for evidence-informed decision making throughout the development process [14]. Moreover, it was observed that LLM can be used to reduce documentation burden by summarizing defect trends and creating testing reports, which can be helpful for managing the test [10].

While the advantages of AI-powered test management are tantalizing, there are also some hurdles. AI predictions rely heavily on high-quality historical data, and incomplete or inconsistent defect repositories result in poor model performance [9]. Moreover, many AI models are “black boxes” that are difficult to understand what they are saying when suggesting. In software engineering contexts,

the challenges that hinder the adoption of AI include explainability, trustworthiness, and transparency [11]. AI governance frameworks are thus crucial to responsible use of AI in test management.

3.2 Test Case Management

Test case management is the development, structuring, updating, running and analysing test cases during the SDLC. Test case management is essential to ensuring sufficient test coverage, regression safety and keeping the test suite relevant as the software changes. The management of test cases in traditional ways is very labour intensive which includes manual writing of test script and periodic manual review of test cases for pruning and updating obsolete cases [1].

AI has significantly revolutionized test case management by automating the creation, selection, and upkeep of test cases. Mostly, the ML algorithms are used for test case generation and refinement to automatically generate test inputs from source code, specification and usage models [2]. Search based software testing (SBST) techniques [7] are based on evolutionary algorithms that are used to create test suites for maximizing coverage criteria, such as branch coverage, and, as shown by Panichella et al., they are able to outperform manually designed test cases.

AI generation of test cases has been expanded to natural language artefacts by natural language processing (NLP) based approaches. It was discovered that using Generative AI models on requirements documents and user stories, test scenarios and acceptance criteria can be automatically generated, which reduces the manual effort needed in the test design phase [11]. This is especially useful in agile settings that have changing requirements and frequently changing test suites.

LLMs have proven themselves to be a powerful generation tool for test cases [10]. The study demonstrated that unit tests, integration test scripts and boundary condition tests can be created from a code context by LLM (specifically GPT-4), and that the quality of the tests is similar to developer written tests in many cases. Their analysis revealed that LLM-driven test generation can save a significant amount of work in manually writing tests, but there are concerns about the quality of the tests generated by the LLM, specifically hallucination, where the model produces logically valid but erroneous statements, which needs to be validated by humans.

AI can also aid in the maintenance and optimisation of test cases. With changes in software, test suites may grow with redundant or obsolete cases [15]. The study demonstrated that search-based techniques are capable of minimizing test suites without losing fault detection power and without compromising the execution time, while preserving the test coverage. Another recent successful application of ML is regression test selection where it can help determine the subset of tests to run when there is a software change, cutting the cost of full regression cycles in CI/CD pipelines.

A fully automatic AI testing framework, which generates, executes, and validates the test cases for Android Apps by utilizing multi-objective search optimisation [8]. The system successfully identified actual faults with only minimal human input, showing the system's potential to be used in industrial environments.

Historically, test libraries have been managed using traditional test case management tools like TestRail, Zephyr, and HP ALM without the help of AI. If there is a change in the UI, AI-powered tools like Testim.io and Mabl involve the use of ML to automatically update the test script, which is another benefit to lower the burden of test script maintenance. These tools embody a blend of legacy test management infrastructure and AI-powered adaptability.

Key challenges in AI-driven test case management are the quality and availability of training data, the potential for test oracle issues (where it's complex to know the correct expected result), and the understanding of AI-generated test cases. According to [2], the empirical testing of the test generation approaches based on ML still has some limitations, and there is a need to explore the generalisability of the approaches across various software domains and languages.

3.3 Defect Management

Defect management involves detecting, recording, categorizing, prioritizing, resolving and tracking software defects during the SDLC. Defect management is also essential for software quality, as defects that are not discovered or those with the wrong priority can lead to expensive production failures. In large-scale, fast-changing software systems, traditional defect management methods based on manual inspection, static analysis tools and heuristic-based prioritisation are becoming inadequate.

AI has had a significant impact on defect prediction and localisation [5]. The study demonstrated that machine learning (ML) models like decision trees, neural networks, Bayesian classifiers and support vector machines are able to classify software components based on their defect-proneness and, therefore, focus testing efforts on high-risk components. These predictive models use source code metrics, change history, developer activity, etc. to generate actionable risk scores.

Defect detection has been further improved with the use of deep learning. Researchers [16] demonstrated that deep neural networks have shown to be more precise and recall than traditional static analysis techniques in discovering complex patterns in source code that are indicative of potential defects. Representations of codes have been used to identify vulnerabilities and logical errors using the convolutional and recurrent neural network architectures, which are able to resist traditional rule-based scanners.

Automated defect classification and triage are also possible with AI. For large software projects, there can be thousands of defect reports that need to be assigned to the correct development team(s) stored in a defect repository [17]. The study demonstrated that using the text of the defect report and the historical assignment trends, ML classifiers can automatically categorize the defect report's content to link it with the developer it is related to, which can help to reduce the time spent on triage and increase the efficiency of solving defects. Unstructured fault reports can also be processed to obtain valuable information with the aid of NLP- based models, such as affected components, severity level, or the reproduction steps.

Another field where AI provides valuable contributions is duplicate defect detection. Duplicate defect detection is yet another domain where AI makes a significant impact. Duplicate bug reports are created in large software projects and take developer time. A study [18] demonstrated that NLP and similarity-based ML models can effectively classify duplicate reports at high precision, thus combining similar incidents and minimizing the duplication of investigation efforts.

With the advent of Generative AI, there are new ways to deal with defects [10]. The study demonstrated that LLMs can assist in writing descriptive defect reports, make root cause hypotheses and suggest candidate fixes, which can speed up the defect resolution cycle. Similarly, the study [11], found that generative AI can offer contextually relevant fix suggestions through reasoning on related code changes and historical defect patterns.

Structured workflow defection tracking tools like Jira, Bugzilla and Azure DevOps Boards do not have predictive or generative features. AI-powered platforms like IBM Watson for DevOps or Seerene embed ML analytics into workflows for resolving defects, with the ability to prioritise them by risk and route them intelligently. The current trend in the defect management is towards these hybrid systems.

The major problems of AI defect management are data quality, the distribution of defective modules is significantly lower than that of modules without defects in defect sets, and cross-project transferability. The study concluded that predictability models built from one project do not necessarily work well on the other project as the technology stack, development practices and team structures differ between the projects. Several methods are proposed to mitigate this, including transfer learning and ensemble methods, which are still research topics.

3.4 Version Management

Defect management is a part of SDLC that includes identifying, reporting, categorizing, prioritizing, fixing and monitoring software defects. Software quality also requires defect management, with defects that are not identified or are given the wrong priorities resulting in costly production failures. For large scale, rapidly changing software systems, the conventional software defect management practices - such as manual inspection and static analysis tools and heuristic-based prioritisation are proving insufficient.

AI has been making a big impact in areas of defect prediction and localisation [5]. The study demonstrated that machine learning (ML) models like decision trees, neural networks, bayesian classifiers and support vector machines can classify software components based on their defect-proneness and, therefore, focus testing efforts on high-risk components. The predictive models are based on source code metrics, change history, developer activity, and more, in order to provide actionable risk scores.

Defect detection has been further improved with the use of deep learning. Deep neural networks have been proved to be more accurate and recall than the traditional static analysis methods for detecting complex patterns in source code that are related to the potential defect [16]. Representations of codes have been used to identify vulnerabilities and logical errors using the convolutional and recurrent neural network architectures, which are able to resist traditional rule-based scanners.

Automated defect classification and triage are also possible with AI. There can be thousands of defect reports in a defect repository that have to be matched to the correct development team(s) when developing a large software project. The content of a defect report can be automatically classified using a text-based ML classifiers based on the defect report description and historical assignment trends for the defect report, as shown in [17], which can contribute to reduce the time needed for the triage process and enhance the efficiency of defect solving. Unstructured fault reports can also be processed to obtain valuable information with the aid of NLP- based models, such as affected components, severity level, or the reproduction steps.

Another field where AI provides valuable contributions is duplicate defect detection. Another area, where AI is having a major impact is in duplicate defect detection. Duplicate bug reports are created in large software projects and take developer time. It has been shown that NLP and similarity-based ML models can be used to effectively classify duplicate reports with high precision [18] and thus to merge similar incidents and to avoid redoing investigation efforts.

With the advent of Generative AI, there are new ways to deal with defects [10]. The study demonstrated that LLMs can assist in writing descriptive defect reports, make root cause hypotheses and suggest candidate fixes, which can speed up the defect resolution cycle. Likewise, [11] discovered that generative AI can provide contextually relevant fixes by reasoning over the related code changes and historical defect patterns.

Structured workflow defection tracking tools like Jira, Bugzilla and Azure DevOps Boards do not have predictive or generative features. AI-powered platforms like IBM Watson for DevOps or Seerene embed ML analytics into workflows for resolving defects, with the ability to prioritise them by risk and route them intelligently. The current trend in the defect management is towards these hybrid systems.

Data quality, low proportion of defective modules in defect sets compared to the proportion of modules without defects, cross project transferability are the main challenges in AI defect management. As the technology stack, development practices and team structures are different between the projects, predictability models developed from one project may not work on the other project, according to [5]. A number of approaches have been recommended to resolve this, such as transfer learning and ensemble methods, both of which are yet to be fully researched.

4. Benefits and Challenges of AI-Driven Testing

The use of AI in software testing brings significant advantages in the following aspects of testing performance, but also poses a set of technical, organisational, and ethical challenges. This review synthesises evidence that allows a structured assessment of both dimensions.

4.1 Benefits

The benefits of AI-powered testing that are found across the board include better test coverage and defects identification. [7] proved that many-objective optimisation techniques can cover all the branches much better than random and manual test generation [8]. The study demonstrated that Sapienz was able to find more faults per unit of testing effort than manual methods in Android application testing. The results of this study align with the literature review and suggest that AI-based test generation covers a wider range of the software state space than traditional approaches.

Another significant benefit is a decrease in the effort and time used for testing [6]. In [6], the study demonstrated that the reduction in time spent for fault detection through selective execution of high-risk tests is possible with the application of ML-based regression test prioritisation [10]. The study demonstrated that the effort of manually creating test cases was significantly lessened with the help of LLM, enabling the testing teams to concentrate on higher-level validation tasks. AI can help streamline the entire testing lifecycle, from designing tests to executing the tests and reporting the results, which can significantly reduce testing cycle times [3], and align with the fast pace of modern CI/CD.

One of the unique advantages of AI is its ability to provide proactive quality assurance by predicting defects. Traditional testing is reactive, meaning it is used to find defects after they are created. As shown by [5] and [13], AI based Defect Prediction Models can be used to move Quality Assurance activities earlier in the development process, which will cut the cost of Defect Resolution. It has long been recognized that the economic value of defect detection early in the development cycle is high and results in much less cost to correct the defect after release [2].

AI also improves consistency and scalability of testing. Human testers can get tired, have cognitive bias, and can have varying quality of tests. At scale, AI systems apply the same logic at all times which can help to test large and complex systems that may be too difficult to test manually [9]. This scalability is especially significant with respect to contemporary software systems that have large combinatorial input spaces.

4.2 Challenges

The biggest problem in AI-Based testing is Data Dependency. However, ML models need vast amounts of quality historical data (including defect data, test execution traces, code metrics, etc.) to make accurate predictions and recommendations [5]. The study demonstrated that the performance of models is significantly affected by the presence of sparse/imbalanced/torn training data sets. Access to AI-driven testing is constrained by the lack of data infrastructure in many organizations, especially smaller companies, which makes it hard to implement effective AI models.

One of the major hurdles to AI in testing scenarios is model opacity. High-performing ML systems like deep neural networks and ensemble methods make recommendations without interpretable reasons. Explainability and transparency [11] are identified as two major challenges that have yet to be resolved, as test managers are not keen on implementing the recommendations of AI if they cannot explain and audit them. The lack of interpretability also makes it more difficult to meet regulatory requirements where written justification for the selection of tests is required.

A new worry for generative AI systems is reliability and hallucination [10]. As identified a common issue with LLM-generated test cases: the likelihood of having statements that are logically correct but

semantically incorrect, which can lead to false sense of security in software quality. The risk of introducing new risks is greater when AI-generated testing artefacts are lacking in robust validation mechanisms. Automated oracles for verifying tests written by AI have yet to be solved and are still a research challenge.

The maturity of integration and tools represent real challenges to the adoption of AI. Research prototypes are highly successful, but integrating these with existing development tools (IDEs, CI/CD, defect tracking tools, etc.) is difficult. There are AI testing tools which need a lot of configuration and domain experience, making them difficult to use for organisations lacking AI expertise.

One of the major challenges with ML-based testing approaches is cross-project generalisability. Differences in development practices, team structure and software architecture between different projects or different technology stacks can lead to models that are trained on one project and not equivalent on another one [5]. This reduces the potential for economies of scale in an investment in AI, because a new model might need to be retrained for each new project context.

5. AI vs Traditional Testing

One of the most critical issues for AI integration in software testing involves the comparison of AI-driven methods and manual/scripted-based automation when it comes to defect detection, test coverage, and efficiency. Comparing AI-driven approaches with manual and scripted automation-based testing in terms of defect detection, test coverage, and efficiency is a central question in the adoption of AI for software testing. The literature searched is analysed in a structured manner, based on the evidence.

5.1 Defect Identification

AI-based testing approaches consistently demonstrate superior defect identification capability compared to traditional methods. It was reported [8], that Sapienz detected a significantly higher number of faults per unit of execution time compared to random testing and human-directed exploratory testing in Android application contexts [5]. Further it was demonstrated that ML-based defect prediction achieves accuracy rates exceeding those of rule-based static analysis tools, particularly for complex defects involving interactions between multiple software components.

Traditional manual testing, while effective for usability and exploratory testing, is limited by human cognitive bandwidth and the inability to systematically explore large input spaces. Scripted automation improves repeatability but is constrained by the foresight of the test designer and fails to detect defects arising from unanticipated input combinations. AI-based generation, by contrast, can explore the software state space more exhaustively, identifying edge cases that human testers would be unlikely to consider [7].

5.2 Test Coverage

An empirical study on the coverage of test generation with AI shows a measurable increase in coverage compared to traditional methods [7]. It was further demonstrated in the study that many-objective optimisation is able to achieve better branch coverage than single-objective evolutionary algorithms and manual testing on various benchmarks software [2], confirmed that test generation tools using ML can provide greater code coverage in a more efficient manner (using fewer test cases), which is a better return on testing investment.

Scripted automation covers only what was designed into the test – it cannot cover what was not thought of in test design. Regression test suites tend to have diminishing coverage as the software evolves, and new code paths are not tested until the regression test suite is manually updated. This limitation can be overcome by using the technique of AI-based approaches; this continuously changes the test generation according to the latest status of the code base [13].

5.3 Test Execution Efficiency

Prioritisation and selection of tests using AI regression analytics provide substantial enhancements in test execution efficiency [6]. It was further demonstrated in the study that prioritisation with ML can significantly shorten the time to the first fault detected, which means that more feedback can be achieved in CI/CD pipelines without running the entire test suite. It is paramount in present development conditions where testing rounds need to be accomplished within a narrow time constraint.

Full regression is very expensive and slow, leading to a build pipeline bottleneck. Although easier and quicker than full regression testing, rule-based testing is not as flexible as ML based testing and can become ineffective for including recently relevant tests if software architecture changes. Artificial intelligence (AI) methods that adapt to historical execution information can be dynamically changed as the software develops, while keeping it efficient in the long run [2].

However, it is important to note that AI-based testing methods come with their own set of costs and requirements, such as the time and computing power needed for model training and inference. In addition to test execution time [9], a key factor to consider when adopting AI is the total cost of ownership (TCO), which encompasses preparation of the data, maintenance of the models, and the effort required for integration. Where there are scarce historical records or software updates are regular, the expense of maintaining AI models might not justify the benefits of execution time.

5.4 Research Findings

Results from our research are reported in Table 1 which compares the performance of traditional methods with that of the AI methods in the four main areas of testing explored in this review, using empirical evidence for each area from Sections 3.1-3.4 as a basis for rating the methods as Low, Medium, and High. The effectiveness factor indicates how well each solution can enable automation, early detection of problems, and scalability to the needs of today's software development landscape.

Testing Domain	Traditional	AI-Driven	Improvement	Key Limitation of AI
Test Management	Low	High	Significant - AI enables predictive analytics, intelligent risk identification, and automated reporting unavailable to manual or rule-based management [9][12]	Gains are conditional on data quality and organisational maturity; immature repositories degrade AI model performance [9]
Test Case Management	Medium	High	Moderate to High - AI automates generation and maintenance at scale; traditional tools require extensive manual authoring and upkeep [7][10]	Hallucination in LLM-generated test cases requires human validation; oracle problems remain unsolved [10][11]
Defect Management	Medium	High	Significant - AI predicts fault-prone modules proactively; traditional approaches are reactive, relying on manual inspection and static rule-based scanners [5][16]	Cross-project generalisation is poor; models must be retrained per project context [5]

Version Management	Low	High	Significant - AI-driven regression selection halved infrastructure cost while maintaining >99.9% fault detection; traditional dependency-based selection scales poorly [20][6]	AI selection may exclude tests that detect unexpected regressions; human oversight remains necessary [14]

Table 1: Comparison of Traditional and AI driven based tools deployment

As shown in Table 1, the test management process in software testing using traditional software testing tools is Low while the AI driven tools are High. This is because traditional test management is reactive and limited to manual planning, expert judgment, and reporting based on rules, while AI-driven tools offer predictive analytics, intelligent risk identification, and automated reporting that adjusts to the project data.

Similarly, for test case management, the table presents that traditional tools are rated as a Medium because test cases need to be manually authored and maintained, whereas AI-based tools are rated as a High, thanks to the search-based technique and LLM for automatically generating, maintaining and optimizing test cases at scale.

For the defect management area, the table indicates that the traditional techniques rate as 'Medium', while static analysis tools and manual inspection are effective only for known defect patterns and are unable to learn from past experiences. Proactive identification of fault prone modules based on Machine Learning classifiers and deep learning models and automated defect triage and duplicate detection are rated High with AI-driven approaches.

Finally, the table indicates that traditional approach to version management is rated Low as dependency-based regression test selection, as a function of the size of the codebase, is difficult to scale and doesn't adapt to evolving software architecture. AI models are given a high rating, as predictive test selection and intelligent change impact analysis significantly cut down execution time without compromising fault detection without increasing it.

6. Future Trends and Research Directions

From the literature analysed, there are various important trends and research challenges that will guide the future of AI in software testing.

Large Language Models and generative AI are the most dynamic and rapidly developing area in AI testing. Both studies [10] and [11], viewed LLMs as a gamechanger which could automate all testing tasks, including requirement analysis, test case generation, defect diagnosis, and test case documentation. But the reliability of LLM outputs and risk of hallucination is a major challenge that needs to be resolved prior to the widespread use of LLMs in the industrial domain. Further research is needed to create automated approaches for validation of LLM-generated testing artefacts such as incorrect assertions and semantically inconsistent test cases.

Explainable AI (XAI) for testing is a research direction of high priority. Many ML models are black-box models, which hinder their credibility and application in test scenarios where decisions need to be traced and explained [11]. Studies on interpretable ML models (IMM) and post hoc explanation methods specific to software testing environments (e.g., explain why a test case was generated or a defect was predicted), would significantly boost practitioners' trust and regulatory compliance.

AI in DevOps and CI/CD is a new research and practice focus [13]. The research further pointed out the need for AI tools that are easily integrated into current development environments. They give

immediate feedback on testing without affecting developer productivity. The study of lightweight AI models, which can learn and adapt to frequent changes in code during the CI/CD environment, would greatly improve the practical application of AI-driven testing.

The generalisation of AI testing models across projects and domains is a key research challenge. Published evaluations tend to be on a single project or a limited domain, which limits generalisability of findings. A study was conducted [2] for more extensive benchmarking work on a wider range of types of software, development processes and programming languages. Federated learning and transfer learning techniques can be potential paths towards models that perform better beyond organizational boundaries, without sharing the data.

The trend towards AI governance and ethics in software testing is a newly identified research area. With the increasing use of AI-generated testing artefacts, such as test cases, defect reports, quality assessments, questions of accountability, bias, and transparency emerge. Organisations need to establish governance structures that establish clear responsibilities for human testers and AI systems, ensure that AI-generated outputs are suitably validated, and avoid the unintended incorporation of systematic bias into quality assurance measures. Developing the research on standards and best practices for responsible use of AI in software testing is valuable for both the academic and industrial world.

Finally, autonomous testing agents represent a long-term research vision. An initial taxonomy of LLM-based testing agents was proposed [21], based on their autonomy it is believed that this autonomy can have a significant impact on the developers who can leverage this autonomy to enable the agents to plan, drive, and evaluate the testing process independently with limited human intervention. As of today, the testing tasks are automatable using AI, but future studies might lead to the development of completely autonomous testing agents that could plan, execute and evaluate a full testing strategy for the software system without human supervision. LLMs, reinforcement learning, and software verification methods might offer a theoretical basis for such systems, but with many challenges in reliability, safety and human oversight.

7.Conclusion

As software systems become more complicated, AI has emerged as an essential tool for software testing, reshaping how businesses prioritise quality assurance. With the growing complexity of software systems, AI has become an irreplaceable asset in software testing, changing the landscape of quality assurance as businesses prioritize it. As traditional, manual, and rule-based testing practices become increasingly ineffective, there is a pressing need for intelligent and scalable alternatives, and AI is emerging to address it. The goal of this research was to analyse and contrast the use of AI based and conventional tools in four key areas: test management, test case management, defect management and version management.

We were able to find the answers to our research questions from our research findings. We found that the applications of AI in software testing span all four major areas examined in this review. AI techniques most frequently used in test case generation and defect prediction are most well-known, whereas test case management and defect management are the most researched areas as described in section 3. The second concern we discovered is that the main benefits of AI-powered testing are increased defect identification, extensive testing coverage, decreased manual work, and proactive quality assurance. Data dependency, model opacity, risks of hallucination in generative systems, and low amount of cross-project generalisation are still the main challenges, while governance and explainability frameworks are described as pre-requisites for responsible adoption in section 4. In our third question, it was concluded that AI-based testing always outperforms traditional manual and scripted automation-based testing in terms of defect identification, test coverage, and test execution efficiency as discussed in section 5. The comparative results in section 3.5 further support the ratings of AI-driven tools as being rated High in all four domains and rates the traditional tools as Low to medium. Answering the last one, it is evident that the current trends and future avenues of research

include the reliability of large language model generated testing artefacts, explainable AI for testing, seamless DevOps and CI/CD integration, cross-project generalisation through transfer and federated learning, and the long-term vision of autonomous testing agents capable of independently planning and executing a complete testing strategy which was explored by Section 6.

AI's future in software testing holds the promise of transforming its capabilities and possibilities. The software testing community is on the brink of a new era of intelligent, proactive and fully scalable quality assurance, as governance frameworks become more mature, explainability grows and AI tools become increasingly available.

References

1. Anand, S., Burke, E. K., Chen, T. Y., Clark, J., Cohen, M. B., Harman, M., Harrold, M. J., McMinn, P., Memon, A., Shen, J., & Yoo, S. (2013). An orchestrated survey of methodologies for automated software test case generation. Journal of Systems and Software, 86(8), 1978–2001.
2. Durelli, V. H. S., Durelli, R. S., Borges, S. S., Endo, A. T., Eler, M. M., Dias, D. R. C., Guimarães, M. P., Magalhães, C. V. C., & Cóbe, R. (2019). Machine learning applied to software testing: A systematic mapping study. IEEE Transactions on Reliability, 68(3), 1189–1212.
3. Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review onapproaches, tools, challenges and practices. IEEE Access, 5, 3909–3943.
4. Kitchenham, B., & Charters, S. (2007). Guidelines for performing systematic literature reviews in software engineering (Technical Report EBSE 2007-001). Keele University and Durham University.
5. Catal, C. (2011). Software fault prediction: A literature review and current trends. Expert Systems with Applications, 38(4), 4626–4636.
6. Marijan, D., Liaaen, M., & Gotlieb, A. (2018). Machine learning for test prioritization: An industrial case study. Software Quality Journal, 26(1), 67–86.
7. Panichella, A., Kifetew, F. M., & Tonella, P. (2015). Reformulating branch coverage as a many-objective optimization problem. In Proceedings of the 8th IEEE International Conference on Software Testing, Verification and Validation (ICST '15) (pp. 1–10). IEEE.
8. Mao, K., Harman, M., & Jia, Y. (2016). Sapienz: Multi-objective automated testing for Android applications. In Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA '16) (pp. 94–105). ACM.
9. Felderer, M., & Ramler, R. (2021). The role of AI in software testing. IEEE Software, 38(3), 104–109.
10. Bavota, G., Linares-Vásquez, M., Bernal-Cárdenas, C., & Di Penta, M. (2024). Large language models for software engineering and testing: A systematic review. ACM Computing Surveys, 57(1), 1–38.
11. Hou, X., Wang, J., Liu, Y., & Li, Z. (2024). Generative AI for software testing: Opportunities and challenges. In Proceedings of the 46th International Conference on Software Engineering (ICSE '24) (pp. 1–12). ACM.
12. Kumar, R., & Mishra, S. (2023). Machine learning-based effort estimation in software testing projects. International Journal of Software Engineering and Knowledge Engineering, 33(4), 567–585.
13. Mäntylä, M. V., Uddin, G., & Khomh, F. (2023). Machine learning applications in continuous testing and DevOps environments. Empirical Software Engineering, 28(5), Article 112.
14. Rahman, M., Ahmed, F., & Karim, M. (2022). Artificial intelligence for software quality management and predictive analytics. Journal of Systems and Software, 190, Article 111340.
15. Harman, M., Mansouri, S. A., & Zhang, Y. (2012). Search-based software engineering: Trends, techniques and applications. ACM Computing Surveys, 45(1), 1–61.

16. Ni, A., Li, N., & Ryu, S. (2022). Just-in-time defect prediction on JavaScript projects: A replication study. In Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22) (pp. 57–68). ACM.
17. Limsettho, N., Hata, H., Ihara, A., & Matsumoto, K. (2014). Automatic unsupervised bug report categorisation. In Proceedings of the 6th Asia-Pacific Symposium on Internetware (pp. 1–9). ACM.
18. Runeson, P., Alexandersson, M., & Nyholm, O. (2007). Detection of duplicate defect reports using natural language processing. In Proceedings of the 29th International Conference on Software Engineering (ICSE '07) (pp. 499–510). IEEE.
19. Yoo, S., & Harman, M. (2012). Regression testing minimization, selection and prioritization: A survey. Software Testing, Verification and Reliability, 22(2), 67–120.
20. Machalica, M., Samylkin, A., Porth, M., & Chandra, S. (2019). Predictive test selection. In Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '19) (pp. 91–10 0). IEEE/ACM.
21. Feldt, R., Kang, S., Yoon, J., & Yoo, S. (2023). Towards autonomous testing agents via conversational large language models. In Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23) (pp. 1688–1693). IEEE.